PrepPressure — API Contract (v3) — FROZEN

> Commit as `docs/API-CONTRACT.md` **before anyone writes code**. Everyone codes against this file. Changing anything here (URL, field name, enum value, error code, collection name, stored field) requires the whole team to agree in a PR that updates this file first. Laith coordinates contract PRs.

Base URL: `/api/v1` (Swagger UI is at `/docs`, outside the prefix).

---

1. Authentication

- Every endpoint requires `Authorization: Bearer <accessToken>` **except** `POST /auth/register` and `POST /auth/login`.
- JWT payload: `{ "sub": <userId number>, "username": "<username>" }`. Secret from `JWT_SECRET`, lifetime from `JWT_EXPIRES_IN` (default `1h`).
- After the global guard verifies the token, every controller can read the current user from the request as:

```
request.user = { userId: number, username: string }
```

This shape is **frozen**. Modules read `request.user.userId` themselves (via `@Req()` or their own small local decorator). They do **not** import anything from the auth module.

- Missing / invalid / expired token → `401` with code `UNAUTHORIZED`.

2. Error format

All errors use NestJS's normal shape plus a stable `code`:

```
{ "statusCode": 409, "error": "Conflict", "code": "QUESTION_IN_USE", "message": "Human readable text" }
```

Validation errors (400) keep Nest's `message` array and use `code: "VALIDATION_FAILED"` where you throw them yourself; errors produced automatically by the global ValidationPipe may lack `code` — the frontend must handle both.

Code	Status	When
`VALIDATION_FAILED`	400	Bad body / query, unknown field
`UNAUTHORIZED`	401	No / bad / expired token
`INVALID_CREDENTIALS`	401	Wrong login
`NOT_OWNER`	403	Editing/deleting/reading something you don't own (where ownership is required)
`NOT_FOUND`	404	Unknown id
`USERNAME_TAKEN` / `EMAIL_TAKEN`	409	Register conflicts
`QUESTION_IN_USE`	409	Delete a question used by an interview or attempt
`QUESTION_NOT_FOUND_IN_LIST`	400	Interview `questionIds` contains an id that doesn't exist
`ATTEMPT_ENDED`	409	Answer/finish on a finished or timed-out attempt
`ATTEMPT_NOT_ENDED`	409	Results requested while attempt in progress
`ANSWER_DUPLICATE`	409	Second answer for the same question
`ANSWER_LATE`	409	Answer after `deadlineAt + 5s` (attempt becomes `timed_out`)
`QUESTION_NOT_IN_ATTEMPT`	400	Answer for a question not in this attempt

Code	Status	When
`ANSWER_TYPE_MISMATCH`	400	Answer type differs from question type
`INVALID_CHOICE`	400	`choiceId` not in the question's choices

3. Shared enums (frozen)

- `difficulty`: `easy` | `medium` | `hard`
- `questionType`: `multiple_choice` | `short_answer` | `code_output`
- `attemptStatus`: `in_progress` | `finished` | `timed_out`
- `readiness`: `needs_practice` | `almost_ready` | `ready`
- `category`: free text, 2–40 chars, stored lowercase and trimmed (e.g. `arrays`, `javascript`)

All dates are ISO 8601 strings in UTC. All ids are positive integers.

---

4. Auth & Users — owner: Badr

POST /auth/register (public)

Body:

```
{ "username": "alex_dev", "email": "alex@example.com", "password": "secret123",
"displayName": "Alex" }
```

Rules: `username` 3–30 chars, `^[a-z0-9_]+$` after lowercasing; valid email; `password` 8–72 chars; `displayName` 2–50. `201` → same body as login response. `409 USERNAME_TAKEN` / `EMAIL_TAKEN`.

POST /auth/login (public)

Body: `{ "usernameOrEmail": "alex_dev", "password": "secret123" }` `200`:

```
{
  "accessToken": "eyJ...",
  "tokenType": "Bearer",
  "expiresIn": 3600,
  "user": { "id": 1, "username": "alex_dev", "email": "alex@example.com",
"displayName": "Alex", "bio": null, "createdAt": "..." }
}
```

`401 INVALID_CREDENTIALS` (same message whether username or password is wrong).

GET /auth/me

`200` → `PrivateUser` (the `user` object above, includes `email`).

PATCH /users/me

Body (all optional): `{ "displayName": "Alex B", "bio": "Backend learner" }` (`bio` max 300, may be `null`). `200` → `PrivateUser`.

GET /users/:id/profile

`200`:

```
{
  "user": { "id": 1, "username": "alex_dev", "displayName": "Alex", "bio": null,
"createdAt": "..." },
  "stats": {
    "questionsCreated": 12,
    "interviewsCreated": 3,
    "attemptsCompleted": 7,
    "attemptsInProgress": 1,
    "questionsAnswered": 41,
    "averagePercentage": 72.4,
    "bestPercentage": 95.0
  }
}
```

`PublicUser` (inside `user`) never includes `email` or `passwordHash`. `404 NOT_FOUND`.

---

5. Questions — owner: Shiam

`PublicQuestion` (everyone):

```
{
  "id": 104, "ownerId": 1, "ownerUsername": "alex_dev",
  "prompt": "What does this print?", "category": "javascript", "difficulty":
"medium", "type": "code_output",
  "choices": null, "codeSnippet": "console.log(1 + '1')",
  "createdAt": "...", "updatedAt": "..."
}
```

For `multiple_choice`, `choices` = `[{ "id": "a", "text": "..." }]` (no `isCorrect`). For others `choices` is `null`. `codeSnippet` is `null` except for `code_output`.

`OwnerQuestion` (only when requester is the owner) = `PublicQuestion` + `choices[].isCorrect`, `expectedKeywords`, `requiredKeywordCount`, `expectedOutput` (null when not relevant).

`ownerUsername` is read from the `users` collection (see section 8) — `null` if the user no longer exists.

GET /questions

Query (all optional): `category`, `difficulty`, `type`, `ownerId`, `search` (substring of prompt, case-insensitive). Unknown query fields → 400. `200` → `PublicQuestion[]` (owner's own questions may come back as `OwnerQuestion`), newest first.

GET /questions/:id

`200` → `OwnerQuestion` if requester is owner, otherwise `PublicQuestion`. `404`.

POST /questions

Body (common + type-specific):

```
{ "prompt": "...", "category": "arrays", "difficulty": "easy", "type":
"multiple_choice",
  "choices": [ { "id": "a", "text": "O(1)", "isCorrect": true }, { "id": "b",
"text": "O(n)", "isCorrect": false } ] }
```

```
{ "prompt": "...", "category": "stacks", "difficulty": "medium", "type":
"short_answer",
  "expectedKeywords": ["lifo", "push", "pop"], "requiredKeywordCount": 2 }
```

```
{ "prompt": "...", "category": "javascript", "difficulty": "hard", "type":
"code_output",
  "codeSnippet": "console.log([1,2].map(x => x * 2))", "expectedOutput": "[ 2, 4
]" }
```

Rules: `prompt` 5–2000; MC: 2–6 choices, choice ids unique 1–10 chars, exactly one `isCorrect: true`; short answer: 2–6 non-empty keywords, `requiredKeywordCount` 1..keywords; code output: `expectedOutput` non-empty, `codeSnippet` 0–5000. Fields that belong to another type → 400. `ownerId` comes from the token, never from the body. `201` → `OwnerQuestion`.

PATCH /questions/:id (owner only)

Body: any subset of the POST fields. The **resulting** question must still be valid (e.g. changing `type` requires the new type's fields). `200` → `OwnerQuestion`. `403 NOT_OWNER`, `404`.

DELETE /questions/:id (owner only)

`204`. `409 QUESTION_IN_USE` if the id appears in any interview's `questionIds` or any attempt's `questions[].questionId`.

---

6. Interviews — owner: Shiam

`InterviewSummary`:

```
{ "id": 7, "ownerId": 1, "ownerUsername": "alex_dev", "title": "Arrays warm-up",
"description": "...",
  "category": "arrays", "difficulty": "easy", "questionCount": 5,
"timeLimitSeconds": 600,
  "createdAt": "...", "updatedAt": "..." }
```

`InterviewDetail` = `InterviewSummary` + `"questions": PublicQuestion[]` (in `questionIds` order; **always** the public form, even for the owner; deleted questions are skipped).

GET /interviews

Query (optional): `category`, `difficulty`, `ownerId`, `search` (in title). `200` → `InterviewSummary[]`, newest first.

GET /interviews/:id

`200` → `InterviewDetail`. `404`.

POST /interviews

```
{ "title": "Arrays warm-up", "description": "Five quick questions", "category":
"arrays", "difficulty": "easy",
  "questionIds": [104, 105, 110], "timeLimitSeconds": 600 }
```

Rules: `title` 3–100; `description` 0–1000; `questionIds` 1–20 distinct integers that all exist (any user's questions may be used) → otherwise `400 QUESTION_NOT_FOUND_IN_LIST`; `timeLimitSeconds` 30–3600. `201` → `InterviewDetail`.

PATCH /interviews/:id (owner only)

Any subset of POST fields, same rules. `200` → `InterviewDetail`. `403`, `404`.

DELETE /interviews/:id (owner only)

`204`. Attempts that used it are not touched.

---

7. Attempts — owner: Esmaeel

`AttemptQuestion` (safe, used while in progress):

```
{ "questionId": 104, "position": 1, "prompt": "...", "type": "multiple_choice",
"difficulty": "easy",
  "category": "arrays", "choices": [ { "id": "a", "text": "O(1)" } ],
"codeSnippet": null }
```

`Attempt`:

```
{ "id": 33, "interviewId": 7, "interviewTitle": "Arrays warm-up", "userId": 2,
  "status": "in_progress", "startedAt": "...", "deadlineAt": "...",
"finishedAt": null,
  "timeLimitSeconds": 600, "questions": [ "AttemptQuestion..." ],
  "answeredQuestionIds": [104] }
```

POST /attempts

Body: `{ "interviewId": 7 }`. `201` → `Attempt`. `404` if interview missing. Copies interview title/time limit and a snapshot of every question (including private scoring data, stored only internally).

GET /attempts

Query (optional): `status`, `interviewId`. Returns **only the current user's** attempts, newest first:

```
[ { "id": 33, "interviewId": 7, "interviewTitle": "Arrays warm-up", "status":
"finished",
    "startedAt": "...", "finishedAt": "...", "score": 3.5, "maxScore": 5,
"percentage": 70.0, "readiness": "almost_ready" } ]
```

(`score` , `maxScore` , `percentage` , `readiness` are `null` while in progress.)

GET /attempts/:id

Owner of the attempt only (`403`). Applies lazy timeout first. `200` → `Attempt` .

POST /attempts/:id/answers

```
{ "questionId": 104, "answer": { "type": "multiple_choice", "choiceId": "a" } }
{ "questionId": 105, "answer": { "type": "short_answer", "text": "A stack is
LIFO: push and pop" } }
{ "questionId": 110, "answer": { "type": "code_output", "text": "[ 2, 4 ]" } }
```

`text` max 2000. `201` → `{ "questionId": 104, "accepted": true, "submittedAt": "..." }` (no correctness revealed). Errors: `ATTEMPT_ENDED` , `ANSWER_LATE` , `ANSWER_DUPLICATE` , `QUESTION_NOT_IN_ATTEMPT` , `ANSWER_TYPE_MISMATCH` , `INVALID_CHOICE` , `403` .

POST /attempts/:id/finish

`200` → `Attempt` with status `finished` . `409 ATTEMPT_ENDED` if already ended.

GET /attempts/:id/results

Owner only. `409 ATTEMPT_NOT_ENDED` while in progress (after applying lazy timeout). `200` :

```
{
  "attemptId": 33, "interviewId": 7, "interviewTitle": "Arrays warm-up",
"status": "finished",
  "startedAt": "...", "finishedAt": "...", "score": 3.5, "maxScore": 5,
"percentage": 70.0, "readiness": "almost_ready",
  "items": [
    { "questionId": 105, "position": 2, "prompt": "...", "type": "short_answer",
      "submitted": { "text": "A stack is LIFO" }, "expected": {
"expectedKeywords": ["lifo","push","pop"], "requiredKeywordCount": 2 },
      "points": 0.5, "maxPoints": 1, "explanation": "Matched 1 of 2 required
keywords (lifo): half credit." }
```

```
  ]
}
```

`submitted` is `null` for unanswered questions. `expected` is `{ "correctChoiceId": "a" }` for MC and `{ "expectedOutput": "..." }` for code output.

---

8. MongoDB storage contract (frozen)

Each module **owns** its collection(s) and is the only one that writes to them. Other modules may **read** them with the native driver (`connection.collection('<name>')`) using only the fields listed here. Public `id` is a numeric field; Mongo's `_id` is never returned.

Collection	Writer	Stored fields other modules may read
`users`	Badr	`id` , `username` , `displayName`
`questions`	Shiam	`id` , `ownerId` , `prompt` , `category` , `difficulty` , `type` , `choices` (`{id,text,isCorrect}`), `codeSnippet` , `expectedKeywords` , `requiredKeywordCount` , `expectedOutput`
`interviews`	Shiam	`id` , `ownerId` , `title` , `questionIds` , `timeLimitSeconds`
`attempts`	Esmaeel	`id` , `userId` , `interviewId` , `status` , `percentage` , `questions` (array; each item has `questionId`), `answers` (array)
`counters`	everyone, own key only	`{ _id: "<key>", seq: number }` with keys `users` , `questions` , `interviews` , `attempts`

Numeric ids: each module writes its **own private helper** that does an atomic `findOneAndUpdate({ _id: '<its key>' }, { $inc: { seq: 1 } }, { upsert: true, returnDocument: 'after' })` on `counters` . Each module touches only its own key, so there is no shared code.

Required indexes (each owner creates their own):

- `users`: unique `username`, unique `email`, unique `id`
- `questions`: unique `id`; `{ category: 1, difficulty: 1, type: 1 }`; `{ ownerId: 1 }`
- `interviews`: unique `id`; `{ ownerId: 1 }`; `{ questionIds: 1 }`
- `attempts`: unique `id`; `{ userId: 1, startedAt: -1 }`; `{ "questions.questionId": 1 }`

9. Frontend shared conventions (frozen)

- API base URL: `import.meta.env.VITE_API_URL` (e.g. `http://localhost:3000/api/v1`).
- Token storage: `localStorage` key **`preppressure.token`**; current user JSON in **`preppressure.user`**.
- Every feature API file sends `Authorization: Bearer <token>` when the token exists.
- On any `401` response, the API file removes both keys and dispatches `window.dispatchEvent(new Event('preppressure:logout'))`. Laith's auth context listens for this event and redirects to `/login`.
- Mocks: when `import.meta.env.VITE_USE_MOCKS === 'true'`, each feature API file returns data from its own `mocks.ts` instead of calling the backend.
- Route paths are the ones in the specification, section 10.